COSTA DE ORO IMPORT

Código clave para la presentación técnica

Backend Spring Boot + servicio de IA en FastAPI

Presentado por	[Integrante 1], [Integrante 2], [Integrante 3]
Asignatura / Docente	[Asignatura] / [Docente]
Ciudad y fecha	Cartagena de Indias, [mes] de 2026

Cómo usar este documento

Cada fragmento indica el **archivo y las líneas exactas** que se muestran en pantalla y **qué explicar**. El código está copiado tal cual del proyecto, con su numeración real. Las rutas de Spring parten de `src/main/java/com/application` y las de FastAPI del repositorio `Costa-De-Oro-Imports-Python`.

Requisito	Sección
Autenticación y gestión de sesiones	1 y 2
Estrategia de consumo de servicios del backend Spring Boot	3
Consumo real de APIs del backend	4
Manejo global de excepciones (funcionalidad avanzada)	5
Procesamiento asíncrono (funcionalidad avanzada)	6
Explicación del modelo de entrenamiento e inferencia en Python	7 y 8
Visualización de resultados generados por IA	9

1. Autenticación y gestión de sesiones (Spring)

Rúbrica: autenticación y gestión de sesiones

Sesiones, recordarme y rutas por rol

`configuration/security/SecurityConfig.java : líneas 52-66`

Qué explicar: Máximo 2 sesiones por usuario con SessionRegistry (la más antigua expira), redirección si la sesión caduca, cookie 'recordarme' firmada y segura, y /admin/** solo para ROLE_ADMIN.

```
52 | .sessionManagement(session -> session
53 |         .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
54 |         .invalidSessionUrl("/auth/login")
55 |         .maximumSessions(2)
56 |         .expiredUrl("/auth/login?expired")
57 |         .sessionRegistry(sessionRegistry()))
58 | .rememberMe(remember -> remember
59 |         .key(rememberMeKey)
60 |         .tokenValiditySeconds(rememberMeValidity)
61 |         .rememberMeParameter("remember-me")
62 |         .useSecureCookie(true))
63 | .authorizeHttpRequests(auth -> auth
64 |         // Configurar endpoints privados
65 |         /* ----- Admin ----- */
66 |         .requestMatchers("/admin/**").hasRole("ADMIN")
```

Cadena de filtros: JWT, reCAPTCHA, formulario y Google

`configuration/security/SecurityConfig.java : líneas 105-111`

Qué explicar: El filtro JWT valida tokens de la API; reCAPTCHA se ejecuta antes del login para frenar bots. El login con formulario y con Google comparten los mismos manejadores de éxito y error.

```
105 | .addFilterBefore(jwtTokenValidatorFilter, BasicAuthenticationFilter.class)
106 | .addFilterBefore(recaptchaFilter, UsernamePasswordAuthenticationFilter.class)
107 | .formLogin(form -> form
108 |         .loginPage("/auth/login")
109 |         .loginProcessingUrl("/auth/login")
110 |         .successHandler(customAuthSuccessHandler)
111 |         .failureHandler(customAuthFailureHandler))
```

2. Autenticación servicio a servicio (FastAPI)

Rúbrica: autenticación · Funcionalidad avanzada: seguridad

API key + permisos + administrador que actúa

`app/api/deps.py : líneas 189-212`

Qué explicar: La API key de Spring se verifica por hash; las lecturas exigen admin:read y las escrituras admin:write. Además X-Admin-Email debe ser un ADMIN activo: la clave sola no basta. Cada acceso queda auditado.

```
189 | try:
190 |     client = repository.verify_api_key(db, api_key)
191 | except repository.HistoryUnavailableError:
192 |     client = None
193 | if client is None:
194 |     log(api_key[:12], "DENEGADO")
195 |     raise HTTPException(401, "API Key inválida o revocada", headers={"WWW-Authenticate": "ApiKey"})
196 |
197 | scopes = {s for s in (client.permisos or "").split(",") if s}
198 | needed = repository.SCOPE_ADMIN_READ if request.method in _SAFE_METHODS else
    |   repository.SCOPE_ADMIN_WRITE
199 | if needed not in scopes and repository.SCOPE_ADMIN_WRITE not in scopes:
200 |     log(f"api:{client.nombre}", "DENEGADO")
201 |     raise HTTPException(403, f"La API Key no tiene el permiso '{needed}'")
202 |
203 | email = (request.headers.get(ADMIN_EMAIL_HEADER) or "").strip()
204 | if not email:
205 |     log(f"api:{client.nombre}", "DENEGADO")
206 |     raise HTTPException(401, f"Falta la cabecera {ADMIN_EMAIL_HEADER} con el administrador que actúa")
207 | user = db.scalar(select(Usuario).where(func.lower(Usuario.correo) == email.lower()))
208 | if user is None or not _valid_account(user) or user_role(user, db) != "ADMIN":
209 |     log(f"api:{client.nombre}->{email[:100]}", "DENEGADO")
210 |     raise HTTPException(403, "El usuario indicado en X-Admin-Email no es un administrador activo")
211 | log(f"api:{client.nombre}->{user.correo}", "PERMITIDO")
212 | return user
```

Claves guardadas con hash y comparación en tiempo constante

`app/prediction/repository.py : líneas 251-261`

Qué explicar: Nunca se guarda la clave en claro; compare_digest evita ataques de tiempo. Una clave revocada deja de servir al instante.

```
251 | def verify_api_key(db: Session, raw_key: str) -> PrediccionApiCliente | None:
252 |     if not raw_key or not raw_key.startswith(API_KEY_PREFIX) or len(raw_key) > 200:
253 |         return None
254 |     try:
255 |         row = db.scalars(select(PrediccionApiCliente).where(
256 |             PrediccionApiCliente.clave_hash == hash_api_key(raw_key))).first()
257 |         if not row or not row.activo or not secrets.compare_digest(row.clave_hash,
    |   hash_api_key(raw_key)):
258 |             return None
259 |         row.ultimo_uso = now()
260 |         db.commit()
261 |         return row
```

Spring envía el administrador de la sesión

`configuration/fastapi/AdminIdentityInterceptor.java : líneas 35-45`

Qué explicar: Solo si el usuario tiene ROLE_ADMIN se añade X-Admin-Email; para anónimos FastAPI responde datos públicos.

```
35 | static String correoAdminActual() {
36 |     Authentication auth = SecurityContextHolder.getContext().getAuthentication();
37 |     if (auth == null || !auth.isAuthenticated() || auth instanceof AnonymousAuthenticationToken) {
38 |         return null;
39 |     }
40 |     boolean esAdmin = auth.getAuthorities().stream().anyMatch(a ->
   |   "ROLE_ADMIN".equals(a.getAuthority()));
41 |     if (!esAdmin) {
42 |         return null;
43 |     }
44 |     String correo = auth.getPrincipal() instanceof UserDetails ud ? ud.getUsername() : auth.getName();
45 |     return correo == null || correo.isBlank() ? null : correo;
```

3. Estrategia de consumo del backend Spring Boot

Rúbrica: estrategia de consumo de servicios

Configuración tipada y validada por entorno

`configuration/fastapi/FastApiProperties.java : líneas 66-82`

Qué explicar: En el perfil cloud la app no arranca si la URL no es https o falta la API key (fail-fast).

```
66 |     if (baseUrl != null) {
67 |         String scheme = baseUrl.getScheme();
68 |         if (scheme == null || baseUrl.getHost() == null) {
69 |             throw new IllegalArgumentException("fastapi.base-url debe ser una URL absoluta (http/https):
   |   " + baseUrl);
70 |         }
71 |         if (requireHttps && !"https".equalsIgnoreCase(scheme)) {
72 |             throw new IllegalArgumentException(
73 |                     "fastapi.base-url debe usar https cuando fastapi.require-https=true (perfil
   |   cloud)");
74 |         }
75 |     }
76 |     // En cloud la credencial es obligatoria; en local se permite vacía para poder arrancar
77 |     // la tienda sin FastAPI (las llamadas devolverán 401 y se registrará el error).
78 |     if (requireHttps && (apiKey == null || apiKey.isBlank())) {
79 |         throw new IllegalArgumentException("FASTAPI_API_KEY es obligatoria en el perfil cloud");
80 |     }
81 |     apiKey = apiKey == null ? "" : apiKey.strip();
82 | }
```

Cliente HTTP: HTTP/1.1, timeouts, cabeceras y errores

`configuration/fastapi/FastApiClientConfig.java : líneas 76-111`

Qué explicar: HTTP/1.1 explícito (bug real con uvicorn), timeouts de conexión y lectura, API key por defecto, interceptores de trazabilidad e identidad, y traducción de 5xx/429 (reintentables) y 4xx (rechazo).

```
 76 | private static RestClient buildClient(FastApiProperties props, Duration readTimeout) {
 77 |     HttpClient httpClient = HttpClient.newBuilder()
 78 |             // HTTP/1.1 explícito: por defecto el cliente de Java intenta "Upgrade: h2c" en http://, que
    |   uvicorn
 79 |             // no soporta, y las peticiones con cuerpo (POST/PUT/PATCH) terminaban en 400 "Invalid HTTP
    |   request".
 80 |             .version(HttpClient.Version.HTTP_1_1)
 81 |             .connectTimeout(props.connectTimeout())
 82 |             .followRedirects(HttpClient.Redirect.NEVER) // nunca reenviar la API key a otro host
 83 |             .build();
 84 |     JdkClientHttpRequestFactory requestFactory = new JdkClientHttpRequestFactory(httpClient);
 85 |     requestFactory.setReadTimeout(readTimeout);
 86 |     return configure(RestClient.builder().requestFactory(requestFactory), props).build();
 87 | }
 88 |
 89 | /** Cabeceras, logging y traducción de errores (separado del transporte para poder testearlo). */
 90 | static RestClient.Builder configure(RestClient.Builder builder, FastApiProperties props) {
 91 |     return builder
 92 |             .baseUrl(props.baseUrl().toString())
 93 |             .defaultHeader(API_KEY_HEADER, props.apiKey())
 94 |             .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
 95 |             .defaultHeader(HttpHeaders.USER_AGENT, "costa-de-oro-spring/1.0")
 96 |             .requestInterceptor(new FastApiLoggingInterceptor())
 97 |             .requestInterceptor(new AdminIdentityInterceptor())
 98 |             .defaultStatusHandler(HttpStatusCode::isError, (request, response) -> {
 99 |                 int status = response.getStatusCode().value();
100 |                 String detail = readBodySnippet(response.getBody());
101 |                 String where = request.getMethod() + " " + request.getURI().getPath();
102 |                 // 429 y 5xx son transitorios -> se reintentan; el resto de 4xx no.
103 |                 if (status == 429 || status >= 500) {
104 |                     throw new FastApiException.Unavailable(
105 |                             "FastAPI respondió " + status + " en " + where + ": " + detail, status,
    |   null);
106 |                 }
107 |                 throw new FastApiException.Rejected(
108 |                         "FastAPI rechazó " + where + " con " + status + ": " + detail, status,
109 |                         FastApiErrorParser.mensaje(detail));
110 |             });
111 | }
```

Reintentos con backoff sin duplicar escrituras

`configuration/fastapi/FastApiGateway.java : líneas 46-65`

Qué explicar: Punto único de salida hacia FastAPI. Una escritura que FastAPI ya recibió no se repite: así no se registra dos veces una factura ni se alterna dos veces un estado.

```
 46 | public <T> T execute(String operation, Function<RestClient, T> call, boolean idempotente) {
 47 |     AtomicInteger attempt = new AtomicInteger();
 48 |     try {
 49 |         return retryTemplate.execute(() -> {
 50 |             int n = attempt.incrementAndGet();
 51 |             try {
 52 |                 return call.apply(restClient);
 53 |             } catch (RuntimeException e) {
 54 |                 RuntimeException toThrow = idempotente || FastApiClientConfig.isConnectFailure(e)
 55 |                         ? e : sinReintentos(e);
 56 |                 if (FastApiClientConfig.isTransient(toThrow)) {
 57 |                     log.warn("FastAPI '{}' intento {} fallido: {}", operation, n, e.getMessage());
 58 |                 }
 59 |                 throw toThrow;
 60 |             }
 61 |         });
 62 |     } catch (RetryException e) {
 63 |         throw translate(operation, attempt.get(), e.getCause() != null ? e.getCause() : e);
 64 |     }
 65 | }
```

Lectura con respaldo local

`configuration/fastapi/FastApiAdmin.java : líneas 61-75`

Qué explicar: Si FastAPI falla, la vista se arma con la lógica local de Spring (misma base de datos), así el panel nunca se cae.

```
61 | private <T> T leer(String operacion, Supplier<T> local, java.util.function.Function<RestClient, T>
   |   llamada) {
62 |     if (!enabled) {
63 |         return local.get();
64 |     }
65 |     try {
66 |         T resultado = fastApi.execute("admin." + operacion, llamada);
67 |         if (resultado != null) {
68 |             return resultado;
69 |         }
70 |         log.warn("FastAPI '{}' devolvió un cuerpo vacío; se usa la lógica local", operacion);
71 |     } catch (FastApiException e) {
72 |         log.warn("FastAPI '{}' no disponible ({}); se usa la lógica local", operacion, e.getMessage());
73 |     }
74 |     return local.get();
75 | }
```

4. Consumo real de APIs del backend

Rúbrica: consumo real de APIs

Controller delgado: datos desde FastAPI

`presentation/controller/admin/ProductoController.java : líneas 52-67`

Qué explicar: El controller solo pide los datos (con respaldo local) y arma la vista; no contiene lógica de negocio.

```
52 | public String DashboardProducto(@AuthenticationPrincipal CustomUserPrincipal principal,
53 |                                  @RequestParam(value = "mensaje", required = false) String mensaje,
54 |                                  @RequestParam(value = "success", required = false) Boolean success,
55 |                                  Model model) {
56 |     Usuario usuario = usuarioService.getUsuarioByCorreo(principal.getCorreo());
57 |     List<ProductoResponse> productoList = fastApiAdmin.leer("producto.listar", "/admin/producto",
58 |             new ParameterizedTypeReference<List<ProductoResponse>>() {
59 |             }, productoService::getProductos);
60 |
61 |     model.addAttribute("usuario", usuario);
62 |     model.addAttribute("urlImagenUsuario", cloudinaryService.getImagenUrl(usuario.getImagen()));
63 |     model.addAttribute("productoList", productoList);
64 |     model.addAttribute("mensaje", mensaje);
65 |     model.addAttribute("success", success);
66 |     return "DashboardProducto";
67 | }
```

Endpoint de FastAPI: dona de ventas por estado

`app/api/v1/endpoints/dashboard_integration.py : líneas 108-125`

Qué explicar: Agrupa las compras por estado: Envíos = PAGADO, Reembolsos = CANCELADO + RECHAZADO, Pedidos = PENDIENTE.

```
108 | def calcular_ventas_totales(db: Session, meses: int = 12, hoy: date | None = None) ->
    |   VentasTotalesResponse:
109 |     """Reparto de las compras por estado (Envíos, Reembolsos, Pedidos) de los últimos ``meses``
    |   meses."""
110 |     anio, mes = ultimos_meses(meses, hoy)[0]
111 |     desde = datetime(anio, mes, 1)
112 |     filas = db.execute(
113 |         select(Compra.estado, func.coalesce(func.sum(Compra.total), 0))
114 |         .where(Compra.fecha >= desde)
115 |         .group_by(Compra.estado)
116 |     ).all()
117 |     por_estado = {str(estado): float(total) for estado, total in filas if estado is not None}
118 |
119 |     series = [round(sum(por_estado.get(e, 0.0) for e in estados), 2) for _, estados in SEGMENTOS_VENTAS]
120 |     total = round(sum(series), 2)
121 |     return VentasTotalesResponse(
122 |         series=series,
123 |         labels=[_formatear_label(nombre, v, total) for (nombre, _), v in zip(SEGMENTOS_VENTAS, series)],
124 |         totalVentas=total,
125 |     )
```

Regla de negocio: la factura suma stock

`app/api/v1/endpoints/admin_extra.py : líneas 466-486`

Qué explicar: Valida número de factura único y que cada producto pertenezca al proveedor; suma el stock en una sola transacción.

```
466 | def crear_factura_proveedor(data: FacturaProveedorRequest, db: DB):
467 |     if db.scalar(select(FacturaProveedor.factura_id).where(FacturaProveedor.numero_factura ==
    |   data.numero_factura)):
468 |         raise error(409, f"Ya existe una factura con el número: {data.numero_factura}")
469 |     _get(db, Usuario, data.proveedor_id, "Proveedor")
470 |     factura = FacturaProveedor(numero_factura=data.numero_factura, fecha_emision=data.fecha_emision,
471 |                                fecha_registro=datetime.now(), total=data.total_compra, activo=True,
472 |                                usuario_id=data.proveedor_id)
473 |     db.add(factura)
474 |     db.flush()
475 |     for d in data.detalles:
476 |         producto = db.scalar(select(Producto).where(Producto.producto_id == d.producto_id,
477 |                                                     Producto.usuario_id == data.proveedor_id))
478 |         if producto is None:
479 |             db.rollback()
480 |             raise error(404, f"Producto no encontrado o no pertenece al proveedor: {d.producto_id}")
481 |         subtotal = d.subtotal if d.subtotal is not None else d.cantidad * d.precio_compra
482 |         db.add(DetalleFactura(cantidad=d.cantidad, precio_compra=d.precio_compra, subtotal=subtotal,
483 |                               factura_id=factura.factura_id, producto_id=producto.producto_id))
484 |         producto.stock = (producto.stock or 0) + d.cantidad
485 |     _commit(db, "No se pudo registrar la factura")
486 |     return ok("Factura de compra registrada exitosamente", 201, id=factura.factura_id)
```

5. Manejo global de excepciones

Funcionalidad avanzada

Manejadores globales en FastAPI

`app/api/v1/endpoints/predictions.py : líneas 206-232`

Qué explicar: Todas las excepciones del servicio responden con un formato único {success, message, error{code, details}}.

```
206 | def install_prediction_error_handlers(app: FastAPI) -> None:
207 |     """Formato de error uniforme en /api/v1/predictions (salvo /demand, que conserva su contrato)."""
208 |     codes = {400: "BAD_REQUEST", 401: "UNAUTHORIZED", 403: "FORBIDDEN", 404: "NOT_FOUND",
209 |              405: "METHOD_NOT_ALLOWED", 409: "CONFLICT", 429: "TOO_MANY_REQUESTS"}
210 |
211 |     @app.exception_handler(PredictionAPIError)
212 |     async def _prediction_error(_request: Request, exc: PredictionAPIError):
213 |         return JSONResponse(status_code=exc.status_code, content=jsonable_encoder({
214 |             "success": False, "message": exc.message, "error": {"code": exc.code, "details":
    |   exc.details}}))
215 |
216 |     @app.exception_handler(StarletteHTTPException)
217 |     async def _http_error(request: Request, exc: StarletteHTTPException):
218 |         if not _is_ml_path(request):
219 |             return await http_exception_handler(request, exc)
220 |         return JSONResponse(status_code=exc.status_code, headers=getattr(exc, "headers", None),
    |   content={
221 |             "success": False, "message": str(exc.detail),
222 |             "error": {"code": codes.get(exc.status_code, f"HTTP_{exc.status_code}"), "details": None}})
223 |
224 |     @app.exception_handler(RequestValidationError)
225 |     async def _validation_error(request: Request, exc: RequestValidationError):
226 |         if not _is_ml_path(request):
227 |             return await request_validation_exception_handler(request, exc)
228 |         details = [{"field": ".".join(str(p) for p in err.get("loc", [])), "message": err.get("msg")}
229 |                    for err in exc.errors()]
230 |         return JSONResponse(status_code=422, content={
231 |             "success": False, "message": "Solicitud inválida", "error": {"code": "VALIDATION_ERROR",
    |   "details": details}})
```

Jerarquía sellada de errores en Spring

`configuration/fastapi/FastApiException.java : líneas 14-36`

Qué explicar: sealed: solo existen 'no disponible' y 'rechazado'; el compilador obliga a tratar ambos casos.

```
 14 | public sealed class FastApiException extends RuntimeException
 15 |         permits FastApiException.Unavailable, FastApiException.Rejected {
 16 |
 17 |     /** Código HTTP devuelto por FastAPI, o {@code null} si no hubo respuesta (fallo de red / timeout).
    |   */
 18 |     private final @Nullable Integer status;
 19 |
 20 |     /** Mensaje para el usuario final que envió FastAPI ({@code detail.mensaje} / {@code detail}), si lo
    |   hay. */
 21 |     private final @Nullable String detalle;
 22 |
 23 |     protected FastApiException(String message, @Nullable Integer status, @Nullable String detalle,
 24 |                                @Nullable Throwable cause) {
 25 |         super(message, cause);
 26 |         this.status = status;
 27 |         this.detalle = detalle;
 28 |     }
 29 |
 30 |     public @Nullable Integer getStatus() {
 31 |         return status;
 32 |     }
 33 |
 34 |     public @Nullable String getDetalle() {
 35 |         return detalle;
 36 |     }
```

6. Procesamiento asíncrono y en segundo plano

Funcionalidad avanzada

Inferencia con semáforo y timeout

`app/prediction/prediction_service.py : líneas 237-248`

Qué explicar: Un semáforo limita las predicciones simultáneas y cada una corre en un hilo con tiempo máximo.

```
237 | if not self._semaphore.acquire(timeout=5):
238 |     raise PredictionBusyError("El servicio de predicción está ocupado; reintenta en unos segundos")
239 | try:
240 |     extraction = load_transactions(db, params.data_source)  # E/S de BD en el hilo de la petición
241 |     future = self._executor.submit(self.forecast, loaded, extraction, steps)
242 |     try:
243 |         result = future.result(timeout=settings.prediction_inference_timeout_seconds)
244 |     except FutureTimeout as exc:
245 |         raise PredictionTimeoutError(
246 |             f"La inferencia superó {settings.prediction_inference_timeout_seconds:.0f} s") from exc
247 | finally:
248 |     self._semaphore.release()
```

Single flight: una sola ejecución para peticiones iguales

`app/api/v1/endpoints/spring_integration.py : líneas 81-96`

Qué explicar: Si llegan varias peticiones iguales, se calcula una vez (candado por clave) y el resto lee la caché.

```
 81 | def cached_prediction(key: str, compute: Callable[[], Any]) -> Any:
 82 |     """Devuelve el valor cacheado o lo calcula una sola vez aunque lleguen peticiones simultáneas."""
 83 |     value = _cache_lookup(key)
 84 |     if value is not None:
 85 |         return value
 86 |     with _lock_for(key):
 87 |         value = _cache_lookup(key)  # otro hilo pudo calcularlo mientras esperábamos
 88 |         if value is not None:
 89 |             return value
 90 |         started = time.perf_counter()
 91 |         value = compute()
 92 |         ttl = settings.prediction_cache_ttl_seconds
 93 |         cache_set_json(key, value, ttl)
 94 |         _local_cache[key] = (time.monotonic() + ttl, value)
 95 |         logger.info("prediccion calculada key=%s en %.1fs (cache %ss)", key, time.perf_counter() -
    |   started, ttl)
 96 |         return value
```

Tarea programada en Spring

`service/implementation/compra/CompraServiceImpl.java : líneas 253-263`

Qué explicar: Cada 2 horas cancela las compras que llevan más de 48 horas pendientes.

```
253 | @Scheduled(fixedRate = 7200000) // 2 hrs
254 | @Transactional
255 | public void limpiarComprasConEstadoPendiente() {
256 |     LocalDateTime hace48Horas = LocalDateTime.now().minusHours(48);
257 |     List<Compra> comprasPendientes = compraRepository.findByEstadoAndFechaBefore(EEstado.PENDIENTE,
    |   hace48Horas);
258 |
259 |     for (Compra compra : comprasPendientes) {
260 |         compra.setEstado(EEstado.CANCELADO);
261 |         compraRepository.save(compra);
262 |     }
263 | }
```

7. Modelo de IA: red neuronal con TensorFlow

Explicación del modelo en Python

Variables sin fuga de información

`app/prediction/preparation.py : líneas 354-379`

Qué explicar: Tendencia, estacionalidad con seno y coseno, y rezagos/medias móviles calculados con shift: ninguna fila usa su propio valor ni el futuro.

```
354 | def build_features(series: pd.DataFrame, cfg: GranularityConfig, origin: pd.Period) -> pd.DataFrame:
355 |     """Variables temporales e históricas. Todas las variables históricas usan ``shift`` para que
356 |     cada fila sólo vea períodos anteriores (sin fuga de información)."""
357 |     frame = series.copy()
358 |     dates = pd.to_datetime(frame["fecha_periodo"])
359 |     frame["periodo_indice"] = [period_offset(p, origin) for p in frame["periodo"]]
360 |     month = dates.dt.month
361 |     frame["mes_sin"] = np.sin(2 * math.pi * (month - 1) / 12)
362 |     frame["mes_cos"] = np.cos(2 * math.pi * (month - 1) / 12)
363 |     if cfg.name == "daily":
364 |         dow = dates.dt.day_of_week
365 |         frame["dia_semana_sin"] = np.sin(2 * math.pi * dow / 7)
366 |         frame["dia_semana_cos"] = np.cos(2 * math.pi * dow / 7)
367 |         frame["es_fin_de_semana"] = (dow >= 5).astype(float)
368 |         frame["dia_mes"] = dates.dt.day.astype(float)
369 |     elif cfg.name == "weekly":
370 |         week = dates.dt.isocalendar().week.astype(float).to_numpy()
371 |         frame["semana_sin"] = np.sin(2 * math.pi * (week - 1) / 52)
372 |         frame["semana_cos"] = np.cos(2 * math.pi * (week - 1) / 52)
373 |     target = frame[TARGET].astype(float)
374 |     for lag in cfg.lags:
375 |         frame[f"ventas_lag_{lag}"] = target.shift(lag)
376 |     previous = target.shift(1)
377 |     frame[f"ventas_media_movil_{cfg.window}"] = previous.rolling(cfg.window,
    |   min_periods=cfg.window).mean()
378 |     frame[f"ventas_desv_movil_{cfg.window}"] = previous.rolling(cfg.window,
    |   min_periods=cfg.window).std(ddof=0)
379 |     return frame
```

Normalización ajustada solo con entrenamiento

`app/prediction/transformation.py : líneas 72-91`

Qué explicar: StandardScaler se ajusta con entrenamiento y se guarda con el modelo para transformar igual al predecir.

```
 72 | def fit(self, train: pd.DataFrame) -> "SalesTensorTransformer":
 73 |     if train.empty:
 74 |         raise TransformationError("No hay datos de entrenamiento para ajustar el scaler")
 75 |     self.x_scaler.fit(self._matrix(train))
 76 |     self.y_scaler.fit(self._target(train))
 77 |     self.fitted = True
 78 |     return self
 79 |
 80 | def _require_fitted(self) -> None:
 81 |     if not self.fitted:
 82 |         raise TransformationError("El transformador no ha sido ajustado")
 83 |
 84 | def transform_features(self, frame: pd.DataFrame) -> np.ndarray:
 85 |     """Transforma datos nuevos con el scaler YA entrenado (nunca se reajusta)."""
 86 |     self._require_fitted()
 87 |     return self.x_scaler.transform(self._matrix(frame)).astype(np.float32)
 88 |
 89 | def transform_target(self, frame: pd.DataFrame) -> np.ndarray:
 90 |     self._require_fitted()
 91 |     return self.y_scaler.transform(self._target(frame)).astype(np.float32)
```

Arquitectura y entrenamiento

`app/prediction/tf_model.py : líneas 54-90`

Qué explicar: Entrada -> Dense(16, ReLU) -> Dense(8, ReLU) -> Dense(1). Adam + MSE; EarlyStopping restaura los mejores pesos.

```
54 | def build_sales_network(n_features: int, learning_rate: float = 0.001, seed: int = DEFAULT_SEED):
55 |     tf = import_tensorflow()
56 |     tf.keras.utils.set_random_seed(seed)
57 |     model = tf.keras.Sequential(name="costa_de_oro_sales_mlp", layers=[
58 |         tf.keras.layers.Input(shape=(n_features,), name="entrada"),
59 |         tf.keras.layers.Dense(16, activation="relu", name="oculta_1"),
60 |         tf.keras.layers.Dense(8, activation="relu", name="oculta_2"),
61 |         tf.keras.layers.Dense(1, activation="linear", name="salida"),
62 |     ])
63 |     model.compile(
64 |         optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
65 |         loss="mean_squared_error",
66 |         metrics=["mean_absolute_error"],
67 |     )
68 |     return model
69 |
70 |
71 | def fit_network(model, data, config: TrainingConfig, on_epoch_end: Callable[[int, dict], None] | None =
   |   None) -> dict:
72 |     """Entrena con EarlyStopping (restaura los mejores pesos) y devuelve el historial por época."""
73 |     tf = import_tensorflow()
74 |     # start_from_epoch evita que el ruido de las primeras épocas (pocos datos) detenga el ajuste
75 |     # antes de que la red aprenda; los mejores pesos se eligen a partir de esa época.
76 |     callbacks = [tf.keras.callbacks.EarlyStopping(
77 |         monitor="val_loss", patience=config.patience, restore_best_weights=True, min_delta=1e-6,
78 |         start_from_epoch=min(config.start_from_epoch, max(0, config.epochs // 2)))]
79 |     if on_epoch_end is not None:
80 |         callbacks.append(tf.keras.callbacks.LambdaCallback(
81 |             on_epoch_end=lambda epoch, logs: on_epoch_end(epoch + 1, dict(logs or {}))))
82 |     history = model.fit(
83 |         data.x_train, data.y_train,
84 |         validation_data=(data.x_val, data.y_val),
85 |         epochs=config.epochs,
86 |         batch_size=max(1, min(config.batch_size, len(data.x_train))),
87 |         shuffle=False,  # respeta el orden temporal; la validación es posterior al entrenamiento
88 |         verbose=0,
89 |         callbacks=callbacks,
90 |     )
```

8. Modelo de IA: Holt-Winters para la gráfica

Explicación del modelo en Python

Nivel y tendencia amortiguada

`app/prediction/timeseries.py : líneas 97-111`

Qué explicar: Nivel y tendencia se actualizan mes a mes sobre la serie sin estacionalidad; phi < 1 aplana la tendencia.

```
 97 | def _run(d: np.ndarray, alpha: float, beta: float, phi: float) -> tuple[np.ndarray, float, float]:
 98 |     """Holt amortiguado sobre la serie desestacionalizada. Devuelve (pronósticos a un paso, nivel,
    |   tendencia)."""
 99 |     n = len(d)
100 |     warm = min(SEASON, n)
101 |     level = float(d[:warm].mean())
102 |     trend = 0.0
103 |     if n >= 2 * SEASON:
104 |         trend = float((d[SEASON:2 * SEASON].mean() - d[:SEASON].mean()) / SEASON)
105 |     preds = np.empty(n)
106 |     for t in range(n):
107 |         preds[t] = level + phi * trend
108 |         prev = level
109 |         level = max(alpha * d[t] + (1 - alpha) * (level + phi * trend), 0.0)  # ventas nunca negativas
110 |         trend = beta * (level - prev) + (1 - beta) * phi * trend
111 |     return preds, level, trend
```

Selección automática de parámetros

`app/prediction/timeseries.py : líneas 114-131`

Qué explicar: Prueba combinaciones de alfa, beta y phi y elige la de menor error de pronóstico a un paso.

```
114 | def fit_holt_winters(y: pd.Series) -> HoltWintersFit:
115 |     """Ajusta el modelo eligiendo alpha/beta/phi por mínimo error cuadrático a un paso."""
116 |     if y.empty:
117 |         raise ValueError("No hay ventas mensuales para ajustar el modelo")
118 |     seasonal = seasonal_indices(y)
119 |     s = seasonal[np.array([p.month for p in y.index]) - 1]
120 |     d = y.to_numpy(dtype=float) / s
121 |     skip = min(SEASON, max(len(d) - 1, 0))  # no puntuar el arranque (el nivel inicial aún no se ha
    |   adaptado)
122 |
123 |     best = None
124 |     for alpha, beta, phi in product(_ALPHAS, _BETAS, _PHIS):
125 |         preds, level, trend = _run(d, alpha, beta, phi)
126 |         sse = float(np.sum((d[skip:] - preds[skip:]) ** 2)) if len(d) > skip else 0.0
127 |         if best is None or sse < best[0]:
128 |             best = (sse, float(alpha), float(beta), float(phi), preds, level, trend)
129 |
130 |     _, alpha, beta, phi, preds, level, trend = best
131 |     residuals = d[skip:] - preds[skip:]
```

Backtest honesto

`app/prediction/timeseries.py : líneas 150-175`

Qué explicar: Entrena sin los últimos 12 meses y los predice. Error: 59 % (método anterior) -> 16 %.

```
150 | def backtest(y: pd.Series, months: int = BACKTEST_MONTHS) -> dict | None:
151 |     """Entrena sin los últimos ``months`` meses y mide el error al pronosticarlos.
152 |
153 |     Se compara con el método ingenuo estacional (repetir el mismo mes del año anterior):
154 |     si el modelo no mejora a ese baseline, la proyección no aporta mucho.
155 |     """
156 |     if len(y) < MIN_MONTHS_BACKTEST:
157 |         return None
158 |     train, test = y.iloc[:-months], y.iloc[-months:]
159 |     model = fit_holt_winters(train)
160 |     predicted = model.forecast(months)
161 |     actual = test.to_numpy(dtype=float)
162 |     naive = y.iloc[-months - SEASON:-SEASON].to_numpy(dtype=float)
163 |     result = {
164 |         "metodo": "backtest",
165 |         "meses": months,
166 |         "desde": str(test.index[0]),
167 |         "hasta": str(test.index[-1]),
168 |         **_error_metrics(actual, predicted),
169 |     }
170 |     result["wape_ingenuo_estacional"] = _error_metrics(actual, naive)["wape"]
171 |     return result
```

9. Visualización de resultados generados por IA

Rúbrica: visualización de resultados de IA

Real, ajuste y proyección

`src/main/resources/static/Js/DashboardPredicciones.js : líneas 67-81`

Qué explicar: Puntos = ventas reales; línea clara = ajuste del modelo; línea discontinua = proyección futura.

```
 67 | function separarSeries(datos) {
 68 |   const reales = [];
 69 |   const ajuste = [];
 70 |   const proyeccion = [];
 71 |   datos.serie.forEach((p, i) => {
 72 |     const x = aFecha(p.fecha);
 73 |     const futuro = p.tipo === 'prediccion';
 74 |     reales.push({ x, y: p.real });
 75 |     ajuste.push({ x, y: futuro ? null : p.prediccion });
 76 |     // El primer punto futuro se une con el último ajuste para que la línea no quede cortada.
 77 |     const siguienteEsFuturo = datos.serie[i + 1]?.tipo === 'prediccion';
 78 |     proyeccion.push({ x, y: futuro || siguienteEsFuturo ? p.prediccion : null });
 79 |   });
 80 |   return { reales, ajuste, proyeccion };
 81 | }
```

Confianza del modelo en pantalla

`src/main/resources/static/Js/DashboardPredicciones.js : líneas 168-174`

Qué explicar: Muestra el error del modelo (WAPE) para que el usuario sepa cuánto confiar en la proyección.

```
168 | function errorMedio(datos) {
169 |   const cerrados = datos.serie.filter((p) => p.tipo === 'real' && p.real != null);
170 |   const totalReal = cerrados.reduce((acc, p) => acc + p.real, 0);
171 |   if (totalReal <= 0) return null;
172 |   const errorAbs = cerrados.reduce((acc, p) => acc + Math.abs(p.prediccion - p.real), 0);
173 |   return (errorAbs / totalReal) * 100;
174 | }
```